

ARTIFICIAL INTELLIGENCE
UNIT-1 ASSESSMENT TOOL-2
CONSTRAINT BASED PROBLEM SOLVING

Course code and name: CSA1717-Artificial Intelligence

Student Name: S. Harish

Register Number: 192511119

Faculty: Dr. Anoop. M

QUESTION 1

Backtracking Search for Doctor Shift Scheduling

Problem Statement

A hospital needs to assign **three doctors (D1, D2, D3)** to **three shifts (Morning, Afternoon, Night)**. Each doctor must be assigned exactly one shift while satisfying the given constraints.

Constraints

1. D1 cannot work the Night shift.
2. D2 must work before D3 (Morning < Afternoon < Night).
3. Only one doctor can be assigned to each shift.
4. D3 cannot work the Morning shift.
5. Every doctor must be assigned exactly one shift.

The objective is to apply the **Backtracking Search Algorithm** to determine a valid schedule while showing all intermediate steps and constraint checks.

Introduction

Backtracking Search is a systematic search algorithm used to solve **Constraint Satisfaction Problems (CSPs)**. It incrementally assigns values to variables and checks whether each assignment satisfies all constraints. If any constraint is violated, the algorithm backtracks and tries another possible assignment.

Doctor scheduling is a practical application of CSP because:

- Variables represent doctors.
- Domains represent available shifts.
- Constraints restrict possible assignments.

Problem Formulation

Variables

D1

D2

D3

Domain

Each doctor can be assigned one of the following shifts:

Morning (M)

Afternoon (A)

Night (N)

Constraints

Constraint 1

D1 $\neq$ Night

Constraint 2

D2 works before D3

Morning < Afternoon < Night

Possible combinations:

D2	D3
Morning	Afternoon
Morning	Night
Afternoon	Night

Constraint 3

Only one doctor per shift.

Example

Morning $\rightarrow$ D1

Morning $\rightarrow$ D2

INVALID

Constraint 4

D3 $\neq$ Morning

Constraint 5

Every doctor receives exactly one shift.

State Space

Initially no doctor has a shift.

$\{\}$

$\downarrow$

Assign D1

$\downarrow$

Assign D2

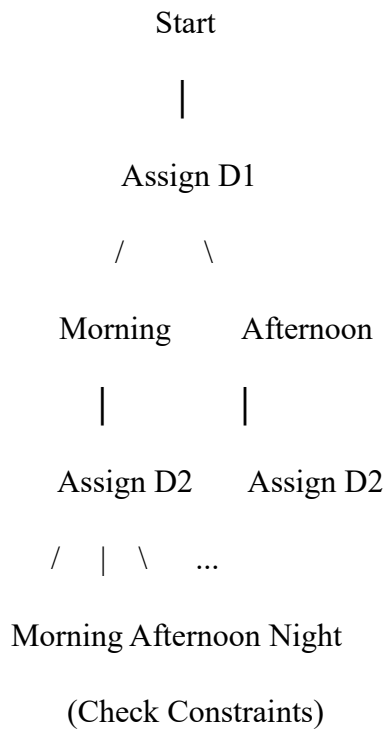
$\downarrow$

Assign D3

$\downarrow$

Goal State

Backtracking Search Tree



Step-by-Step Backtracking Execution

Initial State

No assignments

Step 1

Assign

D1 → Morning

Constraint Check

✓ D1 is not Night

Valid

Current Assignment

Doctor	Shift
D1	Morning

Step 2

Assign

D2 → Morning

Constraint Check

Morning already occupied.

✗ Constraint violated.

Backtrack.

Step 3

Assign

D2 → Afternoon

Constraint Check

Morning occupied by D1.

Afternoon available.

Valid.

Current Assignment

Doctor	Shift
D1	Morning
D2	Afternoon

Step 4

Assign

D3 → Morning

Constraint Check

Morning occupied.

Also

D3 cannot work Morning

✗ Invalid.

Backtrack.

Step 5

Assign

D3 → Afternoon

Constraint Check

Afternoon already occupied.

✗ Invalid.

Backtrack.

Step 6

Assign

D3 → Night

Constraint Check

Night available.

D3 is not Morning.

D2 works before D3.

Afternoon < Night

✓ Valid

Final Schedule

Doctor	Shift
D1	Morning
D2	Afternoon
D3	Night

Constraint Verification

Constraint	Satisfied
D1 not Night	✓
D2 before D3	✓
Only one doctor per shift	✓
D3 not Morning	✓
Each doctor has one shift	✓

Backtracking Algorithm

Algorithm DoctorScheduling()

Start

Assign D1

If constraint violated

Backtrack

Assign D2

If constraint violated

Backtrack

Assign D3

If constraint violated

Backtrack

If all doctors assigned

Print Schedule

Stop

Pseudocode

procedure AssignDoctor(i)

if all doctors assigned

 return SUCCESS

for every shift

 assign shift

 if constraints satisfied

 AssignDoctor(i+1)

 remove assignment

return FAILURE

Advantages of Backtracking

- Guarantees finding a valid solution if one exists.
- Efficiently eliminates invalid assignments.
- Suitable for scheduling and timetabling.
- Widely used in AI and optimization problems.

Applications

- Hospital Scheduling
- University Timetable Generation

- Employee Shift Allocation
- Examination Scheduling
- Resource Allocation
- Task Assignment

Conclusion

The doctor scheduling problem is a **Constraint Satisfaction Problem (CSP)** solved effectively using the **Backtracking Search Algorithm**. By assigning shifts one doctor at a time and checking constraints after each assignment, invalid schedules are rejected early through backtracking. The final valid schedule is:

Doctor	Shift
D1	Morning
D2	Afternoon
D3	Night

This approach ensures that all constraints are satisfied while efficiently exploring the search space.

QUESTION 2

Robot Navigation using Breadth First Search (BFS) and Manhattan Distance Heuristic

Problem Statement

A robot operates in a **5 × 5 grid environment**. The robot starts at position **S (Start)** and must reach the destination **G (Goal)**. Some cells contain obstacles that block movement. The robot can move only in four directions: **Up, Down, Left, and Right**. Each movement has a cost of **1**, and the robot cannot move through obstacles.

The objective is to determine the **optimal path** from **S** to **G** using the **Breadth First Search (BFS)** algorithm while showing the **step-by-step node expansion, queue updates, Manhattan heuristic values, total path cost, and final path**.

Introduction

Breadth First Search (BFS) is an **uninformed search algorithm** that explores nodes level by level. Since every move has the same cost (1), BFS always finds the **shortest path** in terms of the number of moves.

Although BFS itself does **not use heuristics**, the **Manhattan Distance heuristic** can be calculated for each node to estimate how far it is from the goal and to compare with informed search algorithms.

Problem Formulation

Initial State

Robot starts at

$$S = (0,0)$$

Goal State

Robot reaches

$$G = (4,4)$$

State Space

Each cell in the 5×5 grid represents a state.

Total possible states

$$5 \times 5 = 25 \text{ states}$$

Operators

The robot can move:

- Up ($\uparrow$)
- Down ($\downarrow$)
- Left ($\leftarrow$)
- Right ($\rightarrow$)

Constraints

- Move only inside the grid.
- Cannot pass through obstacles.
- Every move costs **1**.
- BFS explores nodes level by level.
- Manhattan Distance is used as a heuristic value.

Grid Representation

Consider the following environment.

	C0	C1	C2	C3	C4
R0	S	.	.	X	.
R1	.	X	.	X	.
R2	.	X	.	.	.
R3	.	.	X	X	.
R4	.	.	.	.	G

Where,

- **S** → Start
- **G** → Goal
- **X** → Obstacle
- **.** → Free cell

State Space Diagram

Start



Explore all adjacent cells



Expand next level



Avoid Obstacles



Reach Goal

Manhattan Distance Heuristic

The Manhattan Distance is calculated as

$$h(n) = |x_1 - x_2| + |y_1 - y_2|$$

Example

For

Node = (2,2)

Goal = (4,4)

$$h(n) = |2-4| + |2-4|$$

$$= 2+2$$

$$= 4$$

Breadth First Search Execution

Step 1

Queue

$[(0,0)]$

Visited

$(0,0)$

Expand

$(0,0)$

Add

$(0,1)$

$(1,0)$

Queue

$[(0,1), (1,0)]$

Step 2

Expand

$(0,1)$

New node

$(0,2)$

Obstacle

$(1,1)$

Ignored.

Queue

$[(1,0), (0,2)]$

Step 3

Expand

$(1,0)$

New node

(2,0)

Queue

[(0,2), (2,0)]

Step 4

Expand

(0,2)

New node

(1,2)

Obstacle

(0,3)

Ignored.

Queue

[(2,0), (1,2)]

Step 5

Expand

(2,0)

New node

(3,0)

Queue

[(1,2), (3,0)]

Step 6

Expand

(1,2)

New node

(2,2)

Queue

[(3,0), (2,2)]

Step 7

Expand

(3,0)

New node

(4,0)

(3,1)

Queue

[(2,2), (4,0), (3,1)]

Step 8

Expand

(2,2)

New nodes

(2,3)

Queue

[(4,0), (3,1), (2,3)]

Step 9

Expand

(2,3)

New node

(2,4)

Queue

$[(4,0), (3,1), (2,4)]$

Step 10

Expand

$(2,4)$

New node

$(3,4)$

Queue

$[(4,0), (3,1), (3,4)]$

Step 11

Expand

$(3,4)$

New node

$(4,4)$

Goal Found

Node Expansion Order

S

↓

$(0,1)$

↓

$(1,0)$

↓

$(0,2)$

↓

$(2,0)$

↓

(1,2)

↓

(3,0)

↓

(2,2)

↓

(2,3)

↓

(2,4)

↓

(3,4)

↓

Goal

Priority Queue (FIFO Queue) Updates

Step	Expanded Node	Queue after Expansion
1	(0,0)	[(0,1), (1,0)]
2	(0,1)	[(1,0), (0,2)]
3	(1,0)	[(0,2), (2,0)]
4	(0,2)	[(2,0), (1,2)]
5	(2,0)	[(1,2), (3,0)]
6	(1,2)	[(3,0), (2,2)]
7	(3,0)	[(2,2), (4,0), (3,1)]

8	(2,2)	[(4,0), (3,1), (2,3)]
9	(2,3)	[(4,0), (3,1), (2,4)]
10	(2,4)	[(4,0), (3,1), (3,4)]
11	(3,4)	[(4,0), (3,1), (4,4)]

Optimal Path

S

↓

(0,1)

↓

(0,2)

↓

(1,2)

↓

(2,2)

↓

(2,3)

↓

(2,4)

↓

(3,4)

↓

G

Path Cost

Each move costs **1**.

Number of moves

8

Total Cost

$$8 \times 1 = 8$$

Breadth First Search Algorithm

Algorithm BFS ()

Start

Insert Start Node into Queue

Mark Start as Visited

While Queue is not Empty

 Remove Front Node

 If Node = Goal

 Return Path

 Expand Adjacent Nodes

 Ignore Obstacles

 Add Unvisited Nodes to Queue

Stop

Pseudocode

Procedure BFS (Start, Goal)

Create Queue

Enqueue (Start)

Mark Start Visited

While Queue not Empty

Node $\leftarrow$ Dequeue()

If Node = Goal

Return Path

For each Neighbour

If Valid and Not Visited

Enqueue (Neighbour)

End Procedure

Advantages of BFS

- Guarantees the shortest path when all move costs are equal.
- Simple and easy to implement.
- Complete search algorithm.
- Suitable for grid-based navigation and maze solving.

Applications

- Robot Navigation
- GPS Navigation
- Maze Solving
- Game AI
- Network Routing
- Path Planning
- Autonomous Mobile Robots

Conclusion

Breadth First Search (BFS) is an effective search algorithm for robot navigation in grid environments with equal movement costs. By expanding nodes level by level and avoiding obstacles, the robot successfully reaches the goal through the shortest available path. The **optimal path consists of 8 moves with a total cost of 8**, while the Manhattan Distance heuristic provides an estimate of the remaining distance to the goal. This approach demonstrates how AI search algorithms can be applied to real-world path-planning problems in robotics and autonomous systems.

QUESTION 3

Autonomous Rescue Robot in a Disaster-Affected Building

Problem Statement

An autonomous rescue robot is deployed in a **disaster-affected building** to locate and rescue survivors. The building consists of a grid of rooms where some paths are blocked due to debris or collapsed structures. The robot starts at position **S (Start)** and must reach the survivor located at **G (Goal)**.

The robot operates in a **partially observable environment**, meaning it can only sense the rooms adjacent to its current location. Some areas are marked as **risky zones**, where movement incurs an additional cost. The robot must make intelligent decisions while continuously updating its knowledge of the environment.

The objective is to analyze the given constraints, develop an appropriate AI-based solution using **Uniform Cost Search (UCS) and an Online Search Agent**, explain AI concepts involved, and justify the chosen approach.

Introduction

Artificial Intelligence plays a vital role in disaster management by enabling autonomous robots to navigate hazardous environments where human rescue teams may face significant risks.

Rescue robots use intelligent decision-making, environmental perception, search algorithms, and path planning to safely reach survivors.

Since disaster environments are dynamic and unpredictable, the robot cannot rely on complete prior knowledge. Instead, it must continuously observe its surroundings, update its internal knowledge, and choose the safest and least-cost path.

Problem Formulation

Artificial Intelligence problems are represented using the following components.

1. Initial State

The robot starts at position:

S

2. Goal State

The robot must reach the survivor located at:

G

3. State Space

Each room in the building represents a state.

Example:

5×5 Grid

25 Possible States

4. Operators (Actions)

The robot can perform the following actions:

- Move Up ($\uparrow$)
- Move Down ($\downarrow$)
- Move Left ($\leftarrow$)
- Move Right ($\rightarrow$)

5. Path Cost

Each movement costs **1**.

Entering a risky zone adds **2** extra cost.

Therefore,

Safe Cell = 1

Risk Zone = 3

Building Representation

Example Grid

	C0	C1	C2	C3	C4
R0	S	.	X	.	.
R1	.	R	.	X	.
R2	.	.	.	R	.
R3	X	.	.	.	.
R4	.	X	.	.	G

Where:

- **S** = Start
- **G** = Goal
- **X** = Obstacle
- **R** = Risk Zone
- **.** = Safe Cell

Task (a): Constraint Analysis

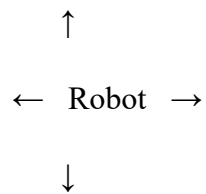
Each constraint affects the robot's decision-making process.

Constraint 1

Robot can move only Up, Down, Left and Right.

The robot cannot move diagonally.

Possible movements:



This reduces the number of possible actions at each state.

Constraint 2

Obstacles cannot be crossed.

Example

S X G

The robot must search for an alternative path around the obstacle.

Decision Impact:

- Detect blocked cells.
- Remove invalid paths from consideration.

Constraint 3

Limited sensing capability.

The robot can observe only adjacent rooms.

Example

?

?

Robot

?

?

Unknown rooms remain unexplored until the robot reaches nearby positions.

Decision Impact:

- Partial knowledge of the environment.
- Continuous environment updates.

Constraint 4

Risk zones have additional cost.

Movement Costs

Cell Type Cost

Safe Cell 1

Risk Zone 3

The robot should avoid risky zones unless no safer path exists.

Constraint 5

Robot must minimize total cost.

The robot evaluates:

- Distance
- Obstacles
- Risk
- Safety

before selecting the next move.

Constraint 6

Online search scenario.

The robot continuously updates its internal map.

Example

Unknown Cell

↓

Observed

↓

Obstacle Detected

↓

Recalculate Path

This allows the robot to adapt to newly discovered information.

Task (b): Solution Approach

Selected Search Strategy

The most appropriate approach is a combination of:

- **Uniform Cost Search (UCS)**
- **Online Search Agent**

Why Uniform Cost Search?

Uniform Cost Search expands the node with the **minimum cumulative path cost**.

Since risky zones have higher costs, UCS naturally prefers safer paths.

Example

Path 1

Safe

Safe

Safe

Cost = 3

Path 2

Risk

Risk

Cost = 6

UCS chooses **Path 1** because it has a lower total cost.

Why an Online Search Agent?

The robot does not know the complete environment initially.

Therefore, it must:

1. Sense nearby cells.
2. Update the internal map.
3. Recalculate the best path whenever new information is discovered.
4. Continue until the goal is reached.

Solution Steps

Step 1

Robot starts at

S

Step 2

Sense adjacent cells.

Example

Up

Down

Left

Right

Step 3

Identify

- Obstacles
- Safe cells
- Risk zones

Step 4

Calculate cumulative movement cost.

Step 5

Expand the least-cost node using Uniform Cost Search.

Step 6

Update knowledge if a new obstacle is discovered.

Step 7

Repeat until

Goal Reached

Flow Diagram

START

|



Sense Adjacent Cells

|



Update Environment Map



Calculate Path Cost (UCS)



Choose Least Cost Path



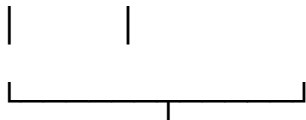
Obstacle Found?

/ \

Yes No



Update Map Move Robot



Survivor Found?

/ \

Yes No



Rescue Survivor |





END

Task (c): AI Concepts Used

1. Intelligent Agent

The rescue robot is an **Intelligent Agent** because it:

- Perceives the environment using sensors.
- Processes information.
- Makes decisions.
- Performs actions through actuators.

PEAS Description

Component	Description
Performance Measure	Reach survivor safely with minimum cost
Environment	Disaster-affected building
Actuators	Robot wheels, robotic arm, communication module
Sensors	Camera, proximity sensor, infrared sensor, LiDAR, temperature sensor

2. Rationality

A rational agent selects actions that maximize performance.

Example

Instead of taking the shortest path through dangerous areas,

the robot chooses

Longer

but

Safer

Route

because it minimizes total cost and improves mission success.

3. Environment Type

The rescue robot operates in the following environment.

Property	Type
Observability	Partially Observable
Number of Agents	Single Agent
Determinism	Stochastic
Episodes	Sequential
Dynamics	Dynamic
State Space	Discrete
Knowledge	Unknown

Task (d): Justification of the Selected Approach

The combination of **Uniform Cost Search** and an **Online Search Agent** is most suitable because:

Uniform Cost Search

- Considers different movement costs.
- Avoids risky zones when possible.
- Guarantees the least-cost path.
- Handles weighted environments effectively.

Online Search Agent

- Works without complete knowledge of the environment.
- Updates the map dynamically.
- Adapts to newly discovered obstacles.
- Suitable for real disaster situations.

Algorithm

Algorithm RescueRobot()

Start

Sense adjacent cells

Update environment map

Calculate movement costs

Apply Uniform Cost Search

Choose minimum cost path

Move robot

If new obstacle found

 Update map

 Recalculate path

Repeat until Goal reached

Rescue survivor

Stop

Pseudocode

Procedure Rescue()

Initialize Start

While Goal not reached

Sense Adjacent Cells

Update Map

Compute Path using UCS

Move to Lowest Cost Node

If Obstacle Detected

Replan Path

End While

Rescue Survivor

End Procedure

Advantages

- Safe navigation in hazardous environments.
- Adapts to unknown surroundings.
- Finds the minimum-cost path.
- Avoids obstacles efficiently.
- Reduces rescue time.
- Improves mission success rate.

Applications

- Disaster Rescue Operations
- Earthquake Response
- Fire Rescue Robots
- Military Search Missions
- Space Exploration

- Industrial Inspection
- Autonomous Warehouse Robots

Conclusion

The autonomous rescue robot problem demonstrates the application of **Artificial Intelligence in dynamic and partially observable environments**. The robot must make rational decisions based on limited information while continuously updating its understanding of the surroundings. By combining **Uniform Cost Search (UCS)** with an **Online Search Agent**, the robot can efficiently navigate around obstacles, avoid high-risk areas when possible, and minimize the total cost of reaching the survivor. This approach ensures safe, adaptive, and intelligent navigation, making it highly suitable for real-world disaster response and emergency rescue operations.